BattingEdge Backend Technical Report

Project: AI-Powered Cricket Shot Analysis & Biomechanics Coaching System

Version: V8p (Production Ready)

Date: December 5, 2025

---

1. Executive Summary

The BattingEdge backend is a high-performance, asynchronous REST API designed to process cricket videos. It integrates Computer Vision (MediaPipe, YOLOv8) and Deep Learning (Bi-LSTM) to perform two distinct tasks simultaneously: **Shot Classification** and **Biomechanical Form Analysis**. The system is built to handle video uploads, real-time inference, and persistent data storage with zero manual intervention.

---

2. Technology Stack

Core Frameworks

- **Language:** Python 3.11+
- **API Framework: FastAPI** (Chosen for speed, async capabilities, and auto-documentation).
- **Server: Uvicorn** (ASGI server for handling asynchronous requests).

AI & Computer Vision

- **TensorFlow/Keras:** Runs the **V8p Bi-LSTM Model** (78% accuracy) for shot classification.
- **MediaPipe Pose:** Extracts 33-point 3D skeletal landmarks from video frames.
- **YOLOv8 (Ultralytics):** Performs object detection to locate the batsman and filter out wicketkeepers/umpires.
- **OpenCV (cv2):** Handles video I/O, frame processing, and drawing the visual overlay (HUD).

Data & Storage

- **SQLite:** Serverless, lightweight relational database to store analysis records.
- **NumPy:** High-performance array manipulation for feature engineering.

- **ReportLab:** Programmatic PDF generation for coaching reports.

---

3. System Architecture & Features

The backend operates on a **Dual-Stream Architecture** to ensure accuracy and visual clarity.

A. Feature 1: Shot Classification (The "Brain")

- **Input:** Raw Full-Frame Video.
- **Process:**
    1. Extracts 33 pose landmarks per frame.
    2. Calculates **4 biomechanical features** (Wrist Velocity, Bat Angle, Knee Angle, Stance Width).
    3. Resamples data to a fixed sequence of **50 frames**.
    4. Feeds 103-dimensional vector into the **V8p Bi-LSTM Model**.
- **Output:** Predicts one of 4 classes: **Drive, Pull, Cut, Sweep** with a confidence score.

B. Feature 2: Biomechanical Form Analysis (The "Coach")

- **Input:** Raw Landmarks from MediaPipe.
- **Logic:** A custom BiomechanicsAnalyzer engine calculates geometry to check 5 key metrics:
    1. **Front Elbow Angle:** Optimal range 120°-140°.
    2. **Head Stability:** Vertical drift < 10cm.
    3. **Back Foot Stability:** Lift < 5cm.
    4. **Hip Rotation:** >30° (Front foot shots) or >60° (Cross bat shots).
    5. **Follow Through:** Hands must finish higher than shoulders.
- **Output:** A Form Score (0-100) and specific, human-readable coaching advice.

C. Feature 3: Visual Overlay Generation (The "Eyes")

- **Logic:** Uses **YOLOv8** to detect the "Tallest Person" (Batsman) to create a focused bounding box.

- **Visuals:** Draws the Skeleton, impact flash, and a Heads-Up Display (HUD) showing the shot type and form score directly on the video.

---

4. Development Workflow: How We Built It

Step 1: The Inference Engine (inference.py)

We started by building the core logic class CricketShotClassifier.

- **Challenge:** The model initially confused "Sweep" with "Drive" because we were cropping the video before processing, destroying the coordinate data.
- **Solution:** We split the logic. We feed the **Full Frame** to the model (to maintain absolute coordinates) but use **Smart Cropping** only for the output video visualization. This restored accuracy from <25% to **75%**.

Step 2: The Database Layer (database.py)

We implemented sqlite3 directly (no heavy ORM) for speed.

- **Schema:** Stores video_id, filename, shot_type, confidence, form_score, and file paths.
- **Functions:** Created atomic functions init_db(), save_initial_upload(), update_analysis_result(), and get_analysis().

Step 3: The API Layer (main.py)

We wrapped the logic in FastAPI endpoints.

- **Async Processing:** Implemented BackgroundTasks. When a user hits /analyze, the server returns "Processing Started" immediately, while the heavy AI work happens in the background. This prevents the UI from freezing.
- **Endpoints Created:**
    1. GET /health: Server status check.
    2. POST /upload: Saves raw video to disk.
    3. POST /analyze/{id}: Triggers the AI pipeline.
    4. GET /result/{id}: Polls for JSON data (Score, Advice).
    5. GET /video/{id}/overlay: Streams the processed MP4.
    6. GET /report/{id}/pdf: Downloads the PDF coaching report.

---

5. Testing & Validation Strategy

We used a rigorous two-phase testing approach to certify the backend for production.

Phase A: Manual Verification (Postman)

- **Method:** We used Postman to simulate a frontend user.
- **Process:**
    1. Manually uploaded a video (drive_test.mp4) via form-data.
    2. Received a UUID.
    3. Triggered the analysis endpoint.
    4. Polled the result endpoint until status: "completed".
    5. Downloaded the binary video response to verify the overlay drawing.
- **Result:** Confirmed that all endpoints handle data correctly and errors (like invalid file types) are caught 400/500 codes.

Phase B: Automated Regression Testing (test_backend.py)

- **Method:** We wrote a Python script using the requests library to automate the entire lifecycle.
- **Workflow:** The script automatically uploads a file, triggers analysis, polls for completion, checks the JSON data for accuracy, and attempts to download the video.
- **Outcome:** All tests passed with green flags (✓ PASS), providing a log file (test_report.txt) as proof of system stability for the defense.

---

6. Conclusion

The BattingEdge backend is now a **fully autonomous system**.

1. **It works automatically:** You upload a video, and the system handles detection, classification, grading, visualization, and reporting without human input.
2. **It is robust:** It handles errors gracefully, validates inputs, and manages large files via streams.

3. **It is connected:** The SQLite database ensures that user data is persistent and can be retrieved anytime.

Status: Ready for Frontend Integration (React).